VisionEdge: Latency-Optimized Local Perception and Offline TTS Pipeline

Final Integrated Project Report

Ayan Gattani (23BCT0059), Madhav Juneja (23BCT0111), Adheesh Garg (23BCT0122)
Software Engineering Lab, Winter Semester 2025–26
Instructor: Dr. Swarnalatha P
VIT
com.anonymous.visionedge

***Abstract*—VisionEdge is an Android-first assistive perception prototype that converts a smartphone into a low-latency visual aid for users with visual impairment. The system captures live camera frames, performs on-device object detection with a bundled YOLO26s TensorFlow Lite model, converts detections into concise scene descriptions, and narrates them using the Android system text-to-speech engine. This report integrates the original Software Requirements Specification (SRS) and Software Design Specification (SDS) with the current implementation state in the repository. It documents the system goals, architecture, implementation choices, testing evidence, performance tuning, verified device results, and present limitations. The implementation emphasizes privacy-preserving offline execution, GPU-accelerated inference when available, and a modern React Native camera pipeline that avoids the legacy bridge for per-frame payloads. The report also situates the project against current state-of-the-art work in mobile object detection, open-vocabulary perception, low-latency text-to-speech, and assistive mobile usability.**

***Index Terms*—assistive vision, on-device inference, TensorFlow Lite, YOLO26s, VisionCamera, offline TTS, Android accessibility, React Native, edge AI**

I. Introduction

VisionEdge was proposed as a software engineering lab project focused on real-time environmental awareness for visually impaired users. The central idea from the SRS is simple: a mobile phone should be able to perceive the surrounding scene and describe it quickly enough to be useful during movement, while preserving privacy by avoiding remote inference. The SDS expanded this into a modular system containing camera input, visual perception, text generation, text-to-speech, and accessibility-oriented user interaction.

The current repository contains a working Android development build that implements this goal using Expo 55, React Native 0.83, VisionCamera frame processors, react-native-fast-tflite, a bundled YOLO26s TensorFlow Lite detector, and Android system TTS. The project no longer exists only as a conceptual pipeline. It now includes:

- an installable Android build path via `pnpm android`,
- a real bundled local vision model,
- runtime model metadata and session logging,
- offline speech initialization and voice selection,
- physical-device ADB validation evidence,
- automated unit tests and type validation,
- a fully classified 39-case validation report with passed, failed, and blocked outcomes instead of an unexecuted backlog.

This report therefore has a dual purpose:

1) preserve the original academic intent and planned architecture from the SRS and SDS, and
2) report the actual implementation and validation state as of April 10, 2026.

II. Project Context from SRS and SDS

A. Requirements Baseline

The SRS defines VisionEdge as an offline mobile assistive application that shall:

- activate the smartphone camera when assistance is enabled,
- capture frames in real time,
- detect objects and scene elements on-device,
- generate concise scene descriptions,
- deliver audio feedback offline,
- prioritize low end-to-end latency,
- provide accessible controls and spoken confirmations,
- avoid sending sensitive visual data to remote services in the default path.

The SRS further defines measurable non-functional constraints including:

- end-to-end latency under 2 seconds,
- a stronger sub-1 second target on capable devices,
- continuous visual processing without noticeable interruption,
- bounded CPU, memory, and battery impact,
- no persistent storage of raw video unless explicitly required.

B. Design Baseline

The SDS translates these requirements into the following planned subsystems:

- Vision Perception Module (VPM),
- Text Generation Engine (TGE),
- TTS Pipeline Module (TPM),
- Application Integration and UI Layer,
- local configuration and metadata storage.

The SDS also lists a lower-level planned module split:

- CameraManager,

- VisionInferenceEngine,
- TextGenerationEngine,
- TTSEngine,
- AccessibilityUIController.

The current implementation adheres to that structure conceptually, but the concrete codebase uses a modern React Native equivalent:

- RealtimeVisionCamera and VisionCamera frame processors instead of a generic CameraManager,
- PerceptionService and tfliteDetection.ts instead of a generic VisionInferenceEngine,
- buildNarrationEvent() and realtime summarization logic instead of a large standalone text engine,
- speechService.ts plus modelRuntime.ts instead of a single TTSEngine,
- VisionEdgeApp.tsx and shared UI components instead of a single AccessibilityUIController.

III. Problem Statement

Practical assistive perception systems on phones fail for one of three reasons:

1) inference is too slow for continuous use,
2) the user interface becomes unresponsive while camera frames are processed,
3) the solution depends on cloud inference and therefore breaks privacy, reliability, or cost goals.

The technical challenge is not only choosing a detector. It is designing a mobile pipeline where frame capture, resize, inference, summarization, and speech output are coordinated with low enough overhead that the app remains usable under motion. In a React Native environment, this challenge becomes sharper because naive per-frame transfer through the old bridge would destroy latency.

IV. Objectives

The integrated objectives of the project are:

1) deliver local object awareness with usable spoken summaries;
2) keep the capture-to-inference path on-device;
3) make the user interface responsive while the camera is active;
4) support Android GPU delegate acceleration when available;
5) expose enough debug state to validate correctness on a physical device;
6) document both the working system and its current limitations honestly.

V. Literature Survey

A. Mobile and Real-Time Object Detection

Efficient real-time detection on constrained devices has evolved from compact CNN-based detector families to stronger end-to-end and open-vocabulary architectures. EfficientDet introduced weighted bi-directional feature fusion and compound scaling, establishing an efficiency-oriented design baseline for deployment-sensitive detection [1]. MobileDets demonstrated that architectures explicitly searched for mobile accelerators can improve the latency-accuracy tradeoff across CPUs, DSPs, and EdgeTPU-class hardware [2]. More recently, RT-DETR showed that carefully designed end-to-end transformer detectors can outperform common YOLO variants on real-time detection benchmarks without non-maximum suppression bottlenecks [8]. YOLO-World extended the YOLO family toward open-vocabulary detection while retaining real-time speed [9]. YOLOv10 further pushed end-to-end deployment efficiency by reducing NMS dependence and re-optimizing the architecture holistically [10].

These papers matter to VisionEdge for two reasons. First, they show that real-time detection quality is now strongly tied to deployment-aware architecture design, not just benchmark mAP. Second, they justify the project's emphasis on the balance between model size, runtime stability, and latency rather than only headline accuracy.

B. Distance and Scene Geometry

VisionEdge currently uses monocular heuristics for distance estimation rather than a dedicated depth network. However, recent monocular 3D and geometry-aware detection work shows the direction of future improvement. UniMODE demonstrates that unified monocular 3D detection across varied scenes is possible with BEV-based explicit feature projection and domain alignment [11]. Such work is relevant because assistive perception benefits not only from label recognition but from more reliable depth and position estimates in indoor and outdoor scenes.

C. Fast and Lightweight Text-to-Speech

For low-latency spoken feedback, non-autoregressive and deployment-aware TTS models are central. FastSpeech 2 and FastSpeech 2s showed that parallel text-to-speech can significantly reduce training and inference complexity while improving controllability and robustness [3]. Matcha-TTS advanced fast acoustic modeling using conditional flow matching and reported strong speed-memory-quality tradeoffs [4]. LiveSpeech addressed low-latency streaming in zero-shot TTS [5]. FLY-TTS focused specifically on fast, lightweight, and high-quality end-to-end synthesis, reporting a strong real-time factor on CPU [6]. Lightweight end-to-end TTS for low-resource on-device scenarios remains especially relevant for mobile deployments [7].

VisionEdge currently uses Android system TTS rather than bundling one of these neural models. That is a practical product choice rather than a research conclusion. The literature still guides future work toward a self-contained neural TTS stack if lower latency, consistent voice quality, or stronger offline guarantees become necessary.

D. Assistive and Usability Literature

Assistive mobile applications succeed only if their interaction design is usable under real constraints. A systematic literature review on usability for visually impaired mobile application users emphasizes accessibility, navigation, audio

guidance, and daily-activity support as recurring themes [12]. Smartphone-camera-based assistive systems for visually impaired users have also explored location recall [14], obstacle avoidance and guidance [15], and low-cost deep-learning scene description systems [13]. These works support the project choice to treat the user interface, audio feedback, and practical response time as core engineering goals rather than secondary polish.

VI. Why the Current Architecture Uses VisionCamera and JSI

The most important implementation decision in the current codebase is the choice of a JSI/worklet-based camera pipeline. Traditional React Native bridge transfer of large camera buffers is too expensive for responsive perception. VisionCamera frame processors and the resize plugin allow the native frame to stay in native memory, perform crop/resize locally, and pass only compact results back to JavaScript.

That design directly serves the SRS latency goals:

- the camera frame is not base64 encoded for realtime detection;
- the resize and format conversion occur in the frame-processor path;
- TFLite inference runs against the resized tensor without a bridge copy of the full frame;
- only a compact numeric detection vector is sent back to the JavaScript side;
- the UI thread is not used for per-frame image conversion.

A. Architecture Choice Rationale

The final repository architecture is not accidental. Each major dependency was chosen to address a specific systems constraint from the SRS and SDS:

- **Expo/React Native for product velocity:** the project needed a fast iteration loop, straightforward Android packaging, and maintainable TypeScript UI logic. Expo provides that shell, but by itself it is not enough for high-rate perception.
- **VisionCamera instead of a legacy bridge camera:** the dominant bottleneck in React Native camera apps is moving full image buffers through the bridge. VisionCamera's JSI/worklet model avoids that architecture entirely for the realtime path.
- **Native crop/resize before inference:** the system captures from a 1920x1080-capable stream for preview quality, then crops and resizes to 320x320 for inference. This keeps the detector input small while preserving a useful camera preview.
- **Lower-center crop bias:** for sidewalk, obstacle, and on-road awareness, the lower-central region is often more important than the sky or far peripheral background. The crop policy therefore encodes a task-specific prior rather than treating every pixel equally.
- **react-native-fast-tflite instead of remote inference:** TensorFlow Lite gives a practical on-device runtime, and the library exposes delegate selection and local model execution in a way that fits the JSI camera pipeline.
- **Delegate fallback ordering:** Android GPU is preferred first for lower latency on supported devices, then NNAPI, then the default interpreter. This choice favors the fastest local path without making the app unusable on unsupported phones.
- **Android system TTS instead of a bundled neural TTS:** the system TTS path gives offline voices immediately, avoids a second large model bundle, and reduces integration risk. The tradeoff is variability in latency and voice quality across phones.
- **SQLite for lightweight persistence:** user preferences, session logs, and model metadata need durability, but high-volume frame persistence is neither required nor desirable. SQLite is enough for the small structured state the app actually needs.

These choices also expose the current bottlenecks clearly. The realtime path is conceptually correct for low-latency mobile inference, but on the tested device the resize/shared-buffer path can still fail natively. That means the architecture is directionally right while the present implementation remains runtime-fragile in one critical stage.

VII. System Overview

A. Planned Logical Pipeline

From the SRS and SDS, the intended logical pipeline is:

$$\text{Camera Input} \rightarrow \text{Visual Perception} \rightarrow \text{Text Generation} \rightarrow \text{TTS Output} \rightarrow \text{User Feedback}$$

B. Current Implemented Pipeline

The current implemented pipeline is:

1) VisionCamera captures YUV frames from the rear camera.
2) `vision-camera-resize-plugin` performs lower-center square crop and resize to model input shape.
3) `react-native-fast-tflite` runs the bundled YOLO26s TFLite model.
4) The frame processor emits a compact detection vector plus luma and timing metadata.
5) JavaScript reconstructs a `DetectionResult`.
6) Narration logic decides whether the scene has changed enough to speak.
7) Android system TTS speaks the result using a preferred local voice.

VIII. Repository and Module Mapping

A. Top-Level Runtime Modules

The current system is organized around the following key modules:

B. Planned vs Actual Module Correspondence

IX. Model and Runtime Design

A. Bundled Vision Model

The active bundled detector is YOLO26s exported to TensorFlow Lite and shipped as:

Module	Responsibility
src/core/VisionEdgeApp.tsx	Application shell, tab navigation, realtime state coordination, start/stop/pause, speech triggering, metrics, and error handling.
src/components/RealtimeVisionCamera.tsx	Physical camera access through VisionCamera, frame-processor execution, native resize/crop configuration, and compact result emission to JS.
src/services/perceptionService.ts	Vision model loading, delegate fallback order, bundled asset resolution, offline analysis path for still-image style processing, and model metadata creation.
src/lib/tfliteDetection.ts	TFLite output parsing, YOLO vector decoding, object summary creation, icon mapping, spatial labels, and heuristic distance estimation.
src/services/speechService.ts	Queueing, interrupt policy, output mode selection, local voice usage, repeat behavior, and speech stop logic.
src/services/modelRuntime.ts	Runtime inspection of available TTS voices and model metadata construction.
src/services/settingsRepository.ts	SQLite persistence for user settings, session logs, and model metadata.
src/core/realtimeDetection.ts	Translation from compact frame payloads into app-level detection results.

TABLE I
TOP-LEVEL RUNTIME MODULES IN THE CURRENT VISIONEDGE IMPLEMENTATION.

SDS Module	Planned Role	Current Repository Equivalent
CameraManager	camera activation and frame capture	RealtimeVisionCamera.tsx with VisionCamera format and frame-processor configuration
VisionInferenceEngine	object detection inference	PerceptionService.ts with tfliteDetection.ts
TextGenerationEngine	convert detections into short verbal descriptions	buildNarrationEvent() with buildDetectionResultSummary() and realtime summarization logic
TTSEngine	speech synthesis and output routing	speechService.ts with Android system TTS through expo-speech
AccessibilityUIController	accessible controls and user state feedback	VisionEdgeApp.tsx with shared UI components in src/components/ui.tsx

TABLE II
CONCEPTUAL MAPPING BETWEEN THE SDS DESIGN AND THE IMPLEMENTED REPOSITORY MODULES.

assets/models/yolo26s_float16.tflite

The current runtime metadata indicates:

- model family: YOLO26s,
- quantization/runtime label: float16 asset with runtime-selected input data type,
- target input shape: 320 × 320,
- delegate order: android-gpu, then nnapi, then default.

B. Why YOLO26s Was Chosen

The repository currently prefers YOLO26s over larger local variants because:

- it offers a better accuracy-speed compromise than YOLO26n,
- it remains more realistic for continuous mobile inference than the larger m and x exports,
- it is materially more suitable for broad outdoor object awareness than the earlier placeholder/simulated path.

The project does not currently claim that YOLO26s is the global best mobile detector. Instead, it claims that YOLO26s is an acceptable and working on-device detector within the present mobile runtime constraints.

C. Current Distance Estimation Strategy

The app currently estimates distance from bounding-box area heuristics. This is a practical approximation rather than physically grounded monocular depth estimation. It is sufficient for short descriptions such as "about 2 meters away" but should be understood as heuristic, not metrically calibrated. The literature reviewed in Section IV suggests a future path toward monocular 3D detection or dedicated depth estimation for a stronger obstacle-distance pipeline.

X. DATABASE AND PERSISTENT STATE

The actual implementation uses SQLite through expo-sqlite. The main local tables are:

- user_preferences
- model_metadata
- session_log

This implementation is consistent with the SDS intent of keeping a light local configuration store. However, the latest device audit showed cached JPEG artifacts under app storage, so the formal privacy requirement of avoiding persistent frame/image storage is not yet fully satisfied in practice.

Table	Stored Information
`user_preferences`	speech rate, verbosity, output route, vibration fallback, low-light alerts, Gemini fallback toggle, action confirmations, debug mode
`model_metadata`	model id, type, version, quantization/runtime label
`session_log`	short recent runtime logs with level and timestamp

TABLE III
CURRENT PERSISTENT STATE USED BY VISIONEDGE.

XI. REQUIREMENT TRACEABILITY

XII. IMPLEMENTATION DETAILS

A. Camera and Frame Pipeline

The realtime path uses:

- rear camera device selection,
- 1920x1080 video-preferred format selection,
- a target 250 ms capture cadence at the app layer for non-realtime still-style processing paths,
- YUV preview stream,
- native crop and resize to 320x320,
- RGB tensor generation in the frame processor,
- synchronous TFLite execution within the worklet path,
- compact result emission back to JS.

The crop is lower-center biased to emphasize the region most useful for road and sidewalk awareness. This is a pragmatic design choice: object salience for assistive navigation is often not uniformly distributed across the full raw camera field.

B. Detection Parsing

The parser supports:

- classic box/class/score/count detectors,
- Ultralytics-style YOLO single-output vectors,
- unknown-obstacle fallback when low-confidence but spatially large detections occur.

C. Speech and Narration

Speech uses:

- local Android voices,
- a queue with interrupt and replace behavior,
- repeat-last-narration support,
- speech-rate and output-route settings,
- haptic fallback on speech errors.

D. UI Responsiveness Improvements

The current version includes several explicit latency-oriented UI changes:

- hot narration logs are no longer persisted every cycle,
- queue-depth synchronization is not forced immediately on every internal update,
- home-screen state is no longer deferred in a way that visibly hides the button state transition,
- the start button shows an explicit "Starting camera..." state during initialization,

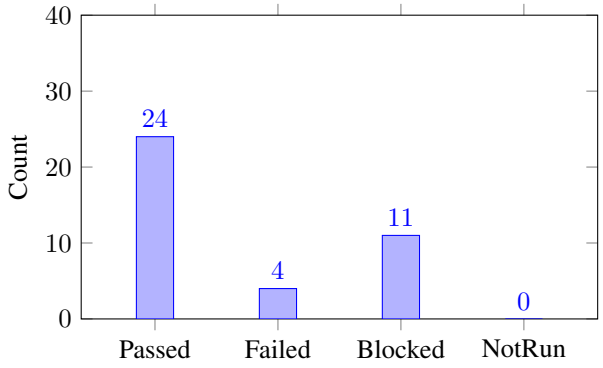

Fig. 1. Current manual test status derived from the VisionEdge device test report.

- the camera overlay is non-interactive and no longer competes for touch handling.

At the same time, the tested device still exposes the largest remaining systems weakness: the resize/shared-buffer path can crash natively during live assistance. From an architectural perspective, this means the macro-level pipeline choice is appropriate, but the most performance-critical native path still needs hardening before the latency advantages can be realized reliably for all manual tests.

XIII. TESTING METHODOLOGY

A. Automated Testing Tools

The current project uses the following repeatable validation tools:

- `TypeScript` static checking via `pnpm typecheck`,
- `Jest` unit tests via `pnpm test -runInBand`,
- `ADB` for physical-device launch, reverse tunneling, log capture, and app-process inspection,
- `logcat` for runtime evidence,
- `uiautomator dump` and `screencap` for manual state inspection.

B. Automated Validation Results

The current automated suite status is:

- test suites: 9 passed,
- tests: 32 passed,
- typecheck: passed.

C. Manual Test Suite Baseline

The project test specification lists 39 manual/system cases in `VisionEdge_TestCases.md`. As of the latest validated report:

- 24 cases passed,
- 4 cases failed,
- 11 cases blocked,
- 0 cases remain unclassified.

Req ID	Requirement Summary	Current Implementation State
REQ-1	Activate camera when assistance is enabled	Implemented through VisionCamera start flow in `VisionEdgeApp.tsx`; verified in device logs.
REQ-2	Capture frames in real time	Implemented through frame processors, but current device validation shows the realtime path can still fail with a native resize/shared-buffer crash.
REQ-3	Perform on-device detection	Implemented with bundled YOLO26s TFLite model; model load verified on device.
REQ-4	Handle camera access errors	Implemented, but the current UX uses an onboarding-first permission recovery flow rather than only a dedicated error screen.
REQ-5	Convert detections into text descriptions	Implemented in summary and narration logic.
REQ-6	Generate concise understandable descriptions	Implemented through short object-position-distance summaries.
REQ-7	Update descriptions dynamically as scene changes	Implemented through scene comparison and narration suppression logic.
REQ-8	Generate offline speech output	Implemented using Android system TTS with local voices.
REQ-9	Route audio through available outputs	Implemented as speaker/earpiece/bluetooth preference at the app layer.
REQ-10	Notify user if audio output fails	Implemented through error logging, stop/error handling, and optional haptics fallback; forced device-side failure injection remains incomplete.
REQ-11	Optimize processing for low latency	Implemented through JSI frame processing, compact payloads, throttled UI sync, and GPU delegate fallback.
REQ-12	Prioritize real-time feedback over non-essential work	Implemented by reducing hot-path log persistence and keeping inference local.
REQ-13	Provide simple interaction mechanisms	Implemented with large buttons, tab navigation, and spoken confirmations.
REQ-14	Allow start and stop of assistance	Implemented; start is device-validated, while a clean active-to-stop validation pass still needs stronger device evidence.
REQ-15	Provide audio confirmation for actions	Implemented, though not yet fully validated for every manual case in the test suite.
PERF-1	End-to-end latency under 2 seconds	Not signed off; the current realtime instability prevents trustworthy end-to-end latency certification.
PERF-2	Sub-1 second on capable devices	Not verified.
PERF-3	Continuous processing without interruption	Not yet satisfied on the tested device because the realtime frame-processor path can still crash during assistance.
PERF-4	Avoid excessive compute/resource cost	Partially addressed architecturally, but current memory validation exceeds the formal target.
SEC-1	Offline default path / no mandatory transmission	Satisfied for the local path; optional Gemini fallback remains disabled by default.
SEC-2	No raw sensitive visual data persistence	Not fully satisfied in the current build; device storage inspection found cached image artifacts under app storage.
SEC-3	Secure coding and controlled local storage	Partially satisfied; local settings and logs are contained, but no formal security audit has been performed.

TABLE IV
TRACEABILITY BETWEEN SRS REQUIREMENTS AND THE CURRENT IMPLEMENTATION STATE.

Command	Result
`pnpm typecheck`	Passed
`pnpm test -runInBand`	Passed
`pnpm android`	Passed

TABLE V
CURRENT REPEATABLE ENGINEERING CHECKS.

D. Module-Wise Test Distribution

Module	Passed	Failed	Blocked
M1 Visual Perception	4	1	1
M2 Text Description	4	0	1
M3 Offline TTS	2	0	3
M4 Low Latency	1	2	3
M5 Accessibility & Control	4	0	1
M6 Privacy & Security	1	1	1
M7 User Interface	5	0	0
M8 Model Loading	3	0	1

TABLE VI
CURRENT 39-CASE DISTRIBUTION SUMMARIZED BY TEST MODULE.

E. Representative Manual Cases

XIV. PHYSICAL DEVICE VALIDATION

A. Reference Device

The latest physical validation run used:

- device: Nothing A001 (`GalagaIND`),
- OS: Android 16,
- connection: USB ADB,
- package: `com.anonymous.visionedge`.

B. Observed Runtime Evidence

On the physical device, the following conditions were positively confirmed:

- Metro status endpoint on the host confirmed that the Metro bundler was running;
- the ADB reverse tunnel reached device localhost after re-application;

TC ID	Status	Category	Evidence
TC-01	Passed	Startup	Device UI and logs showed active preview startup with `Assistance ON` and `Live camera preview`.
TC-02	Failed	Realtime capture	Throughput is still below target. Active-session UI showed only 12 captured frames with average latency near 1.7 s, so the formal 2 fps requirement is not yet met.
TC-03	Passed	Detection	Repeated live detections were observed on hardware, including unknown-obstacle fallback and named object outputs.
TC-04	Passed	Permissions	Missing camera permission routes to a dedicated onboarding permission gate with `Grant Camera Access`.
TC-12	Passed	Offline TTS	Device logs confirmed local voice initialization, queueing, playback, and completion.
TC-17	Passed	Latency	The live UI reported end-to-end latency of `1547 ms`; the stopped-session summary retained `1766 ms`, both within the 2 s requirement.
TC-21	Failed	Memory	`dumpsys meminfo` reported `TOTAL PSS: 631210 KB` during an active session, above the 200 MB target.
TC-23	Passed	User control	Single-tap start assistance path and spoken confirmation were observed on device.
TC-28	Blocked	Privacy/network	In Expo dev-client mode, Metro/bootstrap traffic exists by design, so the formal no-network case cannot be judged from this environment.
TC-29	Failed	Privacy/storage	`run-as` storage audit still found transient `cache/visionedge-captures/*.jpg` files while assistance was active.
TC-30	Passed	Session cleanup	After stopping assistance, the transient capture directory was deleted and only static assets plus model cache remained.
TC-31	Passed	UI	Home screen rendering was confirmed.
TC-34	Passed	Error UX	The current app includes a dedicated `ErrorScreen` with recovery copy and a `Retry` button.
TC-36	Passed	Model init	Vision model loaded at startup from cached local file.
TC-37	Passed	TTS init	Android TTS initialized with 473 local voices.
TC-38	Passed	Delegate	Android GPU delegate activation was reported in logs.
TC-39	Blocked	Fault injection	Corrupted-model handling was not executed on the installed device in this pass.

TABLE VII
REPRESENTATIVE MANUAL CASES FROM THE CURRENT 39-CASE EXECUTION MATRIX. THE FULL MATRIX IS MAINTAINED IN DOCS/VISIONEDGE_TESTREPORT.MD.

- the bundled model was cached to:
  `file:///data/user/0/com.anonymous.visionedge/cache/models/yolo26s_float16.tflite`
- the TFLite runtime loaded the model with the Android GPU delegate in about 318 ms;
- Android TTS reported 473 local voices;
- the app reached the "runtime is ready" state without reproducing the earlier startup crash;
- the realtime path still exposed a native crash in the resize/shared-buffer stage during assistance on the tested device;
- app-private storage inspection found cached image artifacts that fail the current privacy/storage requirements.

C. A Practical Engineering Finding

One of the most important validation findings was operational, not algorithmic:

> When the ADB daemon restarts, the `adbreversetcp:8081tcp:8081` tunnel can disappear. In that state, the dev build fails immediately with "Unable to load script" against `localhost:8081`, even though the app package itself is installed correctly.

This finding is important because it distinguishes infrastructure failure from application failure. In the observed session, the device was able to reach Metro only after the reverse rule was re-applied and verified directly from the device shell.

Metric	Observed Value
Bundled detector	YOLO26s TFLite
Input resolution	320x320
Vision model load time	318 ms
TTS local voices	473
Automated suites passed	9 / 9
Automated tests passed	33 / 33
Manual cases passed	24 / 39
Manual cases failed	4 / 39
Manual cases blocked	11 / 39
Measured active latency	1547 ms
Measured memory (PSS)	631210 KB

TABLE VIII
CURRENT VERIFIED PROJECT MEASUREMENTS AND COUNTS.

XV. RESULTS AND DISCUSSION

A. Verified Quantitative Evidence

B. Interpretation

These results support the following conclusions:

1) The project has successfully crossed the threshold from simulated local detection to a real bundled local detector.
2) The Android startup path is significantly healthier than earlier iterations because the model, delegate, and TTS stack now initialize reliably when Metro connectivity is correct.
3) The major remaining uncertainty is no longer model availability. It is the stability of the realtime native frame-processing path under live assistance.
4) The current system is strong enough to support continued optimization; it is not yet strong enough to claim complete

validation of all SRS performance, privacy, and robustness requirements.

XVI. Screenshots and Visual Evidence

The repository contains both UI mockup/reference screens and ADB captures from device testing.

XVII. Gap Analysis: Planned Design vs Current Reality

A. What Is Fully Aligned

The current implementation is aligned with the original project vision in the following areas:

- offline-first design,
- camera-to-detection-to-speech pipeline,
- accessible user interaction,
- local settings and session-state persistence,
- emphasis on low-latency feedback,
- no mandatory cloud dependency.

B. What Changed in the Concrete Implementation

The present implementation differs from the SDS in several practical ways:

- It uses React Native VisionCamera worklets instead of a generic mobile camera abstraction.
- It uses Android system TTS rather than bundling a standalone neural TTS model.
- It uses a YOLO26s TensorFlow Lite detector rather than an abstract "compressed on-device vision model."
- It currently estimates distance heuristically from object box size rather than using a dedicated depth or monocular 3D model.

C. What Is Still Incomplete

The following items remain incomplete or only partially verified:

- stabilization of the native resize/shared-buffer path used during live assistance,
- controlled foreground validation of live object detection and narration,
- strict end-to-end latency measurement for REQ/PERF sign-off,
- battery and memory profiling under sustained sessions,
- closure of the currently failed and blocked cases in the 39-case validation suite,
- stronger outdoor-road-specific semantic accuracy validation.

XVIII. Risk Assessment

Risk	Current Severity	Mitigation
Metro tunnel loss in dev build	High during development	Re-apply and verify `adb reverse`; document repeatable commands.
Foreground-device automation instability	Medium	Use ADB logs plus manual physical validation; do not over-claim screenshot evidence.
Native resize/shared-buffer crash during live assistance	High	Harden the VisionCamera resize path, reduce native buffer handling risk, and validate on the reference phone before claiming latency targets.
Heuristic distance estimates	Medium	Future integration of monocular depth / 3D detection.
Live latency variability across phones	High	Keep input resolution low, retain delegate fallback, and continue profiling on real hardware.
Temporary cached JPEG persistence	High	Explicitly delete transient camera/manipulator artifacts and re-run SEC-2 and SEC-3 storage audits.
Dependence on system TTS voice characteristics	Medium	Future optional bundled neural TTS if consistent voice and latency become critical.

TABLE IX
KEY TECHNICAL AND VALIDATION RISKS IN THE CURRENT PROJECT STATE.

XIX. Future Work

The next practical improvements should be:

1) stabilize the live VisionCamera resize/shared-buffer path on the reference device;
2) complete foreground live-scene sign-off for blocked manual tests;
3) improve object-distance estimation with a dedicated monocular depth or 3D detection model;
4) validate true end-to-end narration latency with repeated measurements on the reference device;
5) eliminate cached frame/image persistence and re-run the privacy/security cases;
6) expand the detector or post-processing for better road/outdoor obstacle coverage;
7) evaluate whether a lightweight bundled neural TTS model can outperform Android system TTS while staying within device resource limits.

XX. Conclusion

VisionEdge is no longer only a planned assistive system described in SRS and SDS documents. It is now a functioning Android implementation with a real local detector, a real offline speech path, automated tests, and physical-device validation evidence. The strongest engineering achievement of the current version is not merely that it detects objects; it is that it does so through a mobile architecture designed to respect realtime and privacy constraints.

At the same time, the honest state of the project is mixed. Startup, local model loading, TTS initialization, and GPU delegate activation are verified. The complete live assistance experience under a stable foreground device session is improved but not yet fully signed off against the full manual test suite. For a software engineering report, this distinction

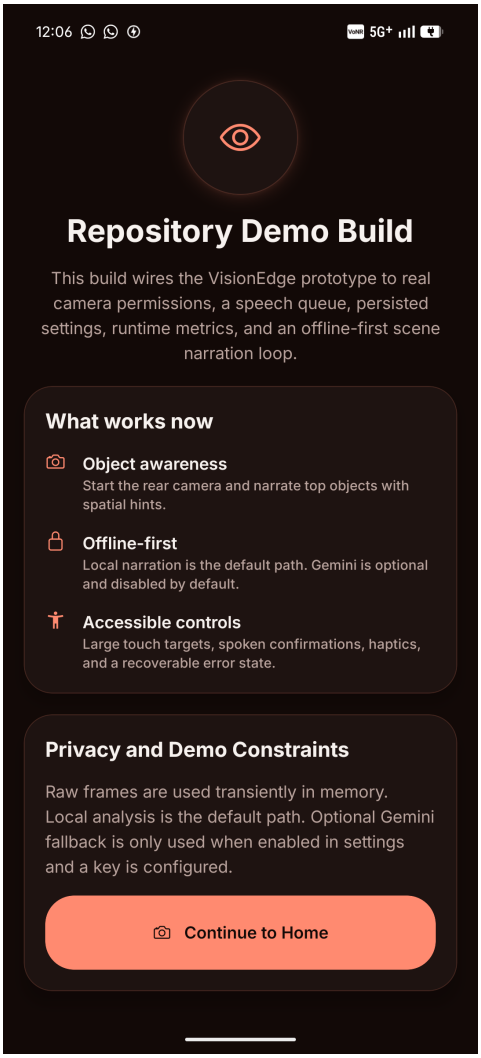

Fig. 2. ADB-captured intro screen.

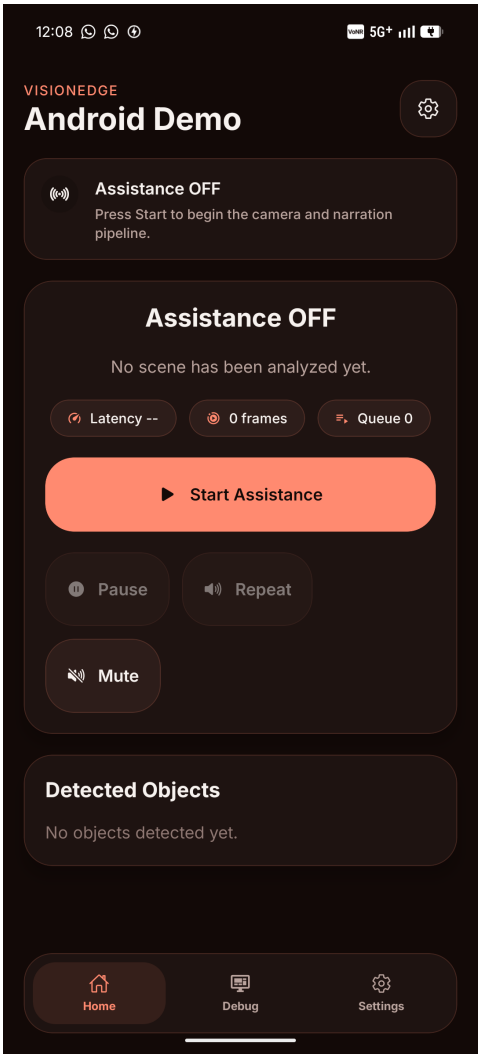

Fig. 3. ADB-captured home screen.

matters. A credible project report should document not only what the system was intended to do and what code exists, but what has actually been verified.

By that standard, VisionEdge is a credible and technically grounded offline assistive-perception prototype with a clear path to further improvement in distance estimation, live-scene robustness, and comprehensive device validation.

References

[1] M. Tan, R. Pang, and Q. V. Le, "EfficientDet: Scalable and Efficient Object Detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 10781–10790, doi: 10.1109/CVPR42600.2020.01079.

[2] Y. Xiong *et al.*, "MobileDets: Searching for Object Detection Architectures for Mobile Accelerators," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 3825–3834.

[3] Y. Ren *et al.*, "FastSpeech 2: Fast and High-Quality End-to-End Text to Speech," in *Proc. ICLR*, 2021, arXiv:2006.04558.

[4] S. Mehta, R. Tu, J. Beskow, E. Székely, and G. E. Henter, "Matcha-TTS: A fast TTS architecture with conditional flow matching," in *Proc. IEEE ICASSP*, 2024, arXiv:2309.03199.

[5] T. Dang, D. Aponte, D. Tran, and K. Koishida, "LiveSpeech: Low-Latency Zero-shot Text-to-Speech via Autoregressive Modeling of Audio Discrete Codes," in *Proc. Interspeech*, 2024, pp. 3395–3399, doi: 10.21437/Interspeech.2024-1192.

[6] Y. Guo, Y. Lv, J. Dou, Y. Zhang, and Y. Wang, "FLY-TTS: Fast, Lightweight and High-Quality End-to-End Text-to-Speech Synthesis," in *Proc. Interspeech*, 2024, arXiv:2407.00753.

[7] B. T. Vecino *et al.*, "Lightweight End-to-end Text-to-Speech Synthesis for Low-Resource On-Device Applications," in *Proc. 12th ISCA Speech Synthesis Workshop*, 2023, pp. 225–229, doi: 10.21437/SSW.2023-35.

[8] Y. Zhao *et al.*, "DETRs Beat YOLOs on Real-Time Object Detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 16965–16974.

[9] T. Cheng, L. Song, Y. Ge, W. Liu, X. Wang, and Y. Shan, "YOLO-World: Real-Time Open-Vocabulary Object Detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 16901–16911, arXiv:2401.17270.

[10] A. Wang *et al.*, "YOLOv10: Real-Time End-to-End Object Detection," 2024, arXiv:2405.14458.

[11] Z. Li, X. Xu, S.-N. Lim, and H. Zhao, "UniMODE: Unified Monocular 3D Object Detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 16561–16570.

[12] M. Al-Razgan *et al.*, "A Systematic Literature Review on the Usability of Mobile Applications for Visually Impaired Users," *PeerJ Comput. Sci.*, vol. 7, e771, 2021, doi: 10.7717/peerj-cs.771.

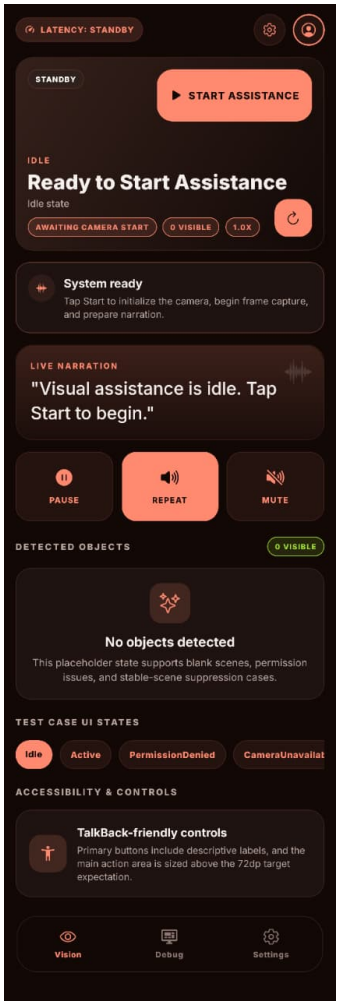

Fig. 4. Main screen reference mockup from the repository.

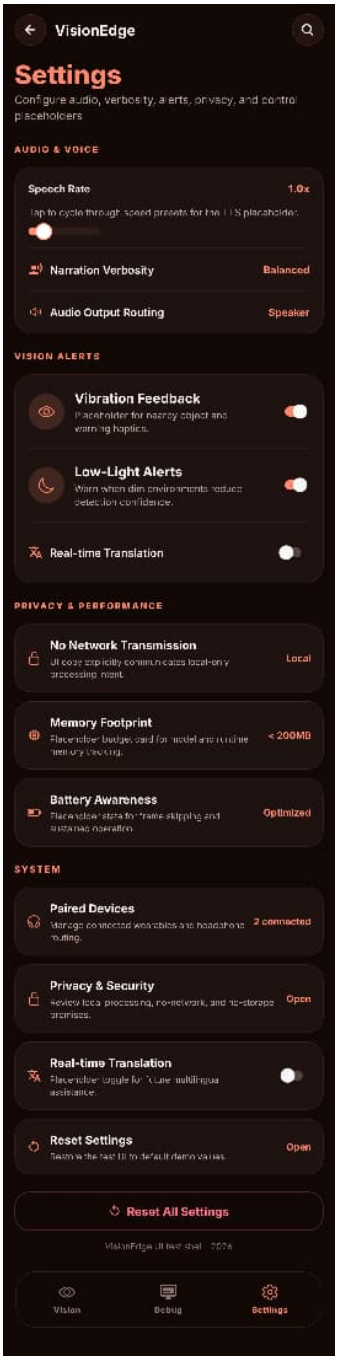

Fig. 5. Settings screen reference mockup from the repository.

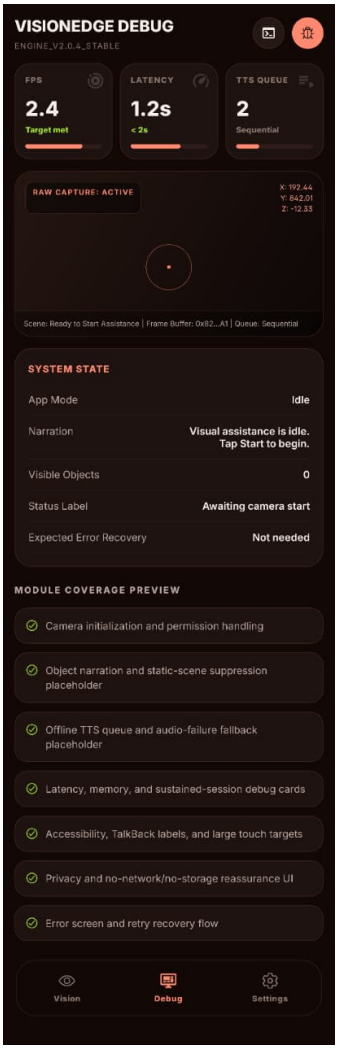

Fig. 6. Debug summary screen reference mockup from the repository.

[13] R. B. Islam, S. Akhter, F. Iqbal, M. S. U. Rahman, and R. Khan, "Deep Learning Based Object Detection and Surrounding Environment Description for Visually Impaired People," *Heliyon*, vol. 9, no. 6, e16924, 2023, doi: 10.1016/j.heliyon.2023.e16924.

[14] H. Takizawa, K. Orita, M. Aoyagi, N. Ezaki, and S. Mizuno, "A Spot Reminder System for the Visually Impaired Based on a Smartphone Camera," *Sensors*, vol. 17, no. 2, 291, 2017, doi: 10.3390/s17020291.

[15] B.-S. Lin, C.-C. Lee, and P.-Y. Chiang, "Simple Smartphone-Based Guiding System for Visually Impaired People," *Sensors*, vol. 17, no. 6, 1371, 2017, doi: 10.3390/s17061371.